

Résumé du Projet SkillMatch

Introduction

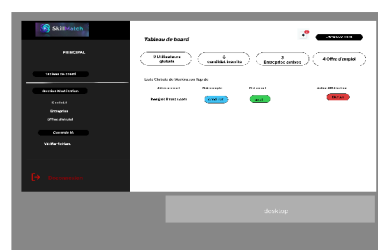
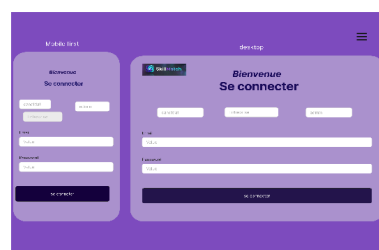
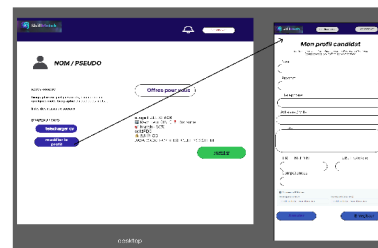
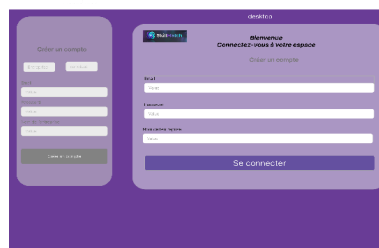
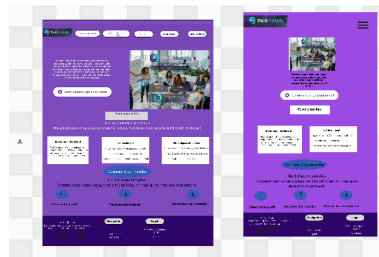
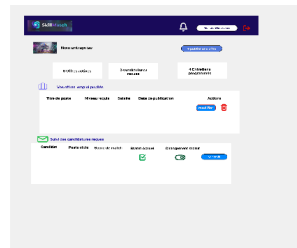
SkillMatch est une plateforme web de mise en relation entre candidats et recruteurs dans le secteur des technologies de l'information, conçue comme projet de validation du titre professionnel Développeur Web et Web Mobile (DWWM). L'objectif principal est de résoudre la problématique du "bruit" dans le recrutement IT, où les recruteurs reçoivent quotidiennement des flux massifs de CV non structurés, rendant l'identification des compétences techniques spécifiques longue et sujette à l'erreur.

La solution apportée est une application web centralisée où l'entreprise publie ses offres avec des mots-clés de compétences attendues, et où le candidat soumet son profil. L'application croise ces jeux de données textuels et génère un indicateur de pertinence en temps réel via un algorithme de matching sémantique.

ACTIVITÉ TYPE 1 : Développer la partie front-end d'une application web ou web mobile sécurisée

CP 1 : Maquetter une application

- Design System & Charte Graphique : Création d'une identité visuelle professionnelle dominée par des bleus profonds pour asseoir la dimension institutionnelle et RH. Les touches fonctionnelles s'appuient sur un vert émeraude (succès, score de matching élevé, boutons primaires) et un orange de sécurité (alertes modérées, profils intermédiaires).
- Processus de conception : Conception en amont de l'ensemble de l'application via l'outil Figma (conception des wireframes et des maquettes haute fidélité). Définition claire des User Stories pour structurer la navigation selon trois profils types : le Candidat, l'Entreprise (recruteur) et l'administrateur.



CP 2 : Réaliser une interface utilisateur web animée et responsive

- Approche Mobile-First : Grâce au système de grille et aux breakpoints de Tailwind CSS, l'application adopte une stratégie Mobile First.
- Les tableaux de données complexes sur Desktop se structurent en "cartes" lisibles sur Mobile, permettant aux recruteurs de consulter les profils en déplacement.
- Dynamisme visuel : Intégration de Bootstrap pour la mise en page et les composants pré-stylisés. Création de composants graphiques modernes :

- Indicateurs de matching : Affichage du score de correspondance sous forme de badge numérique, permettant une lecture rapide par le recruteur. Une version avec barres de progression et code couleur (vert pour les scores élevés, orange pour les scores moyens, rouge pour les scores faibles) est prévue en évolution future.
- Toasts lors des uploads de fichiers.
- Modales de confirmation pour les actions critiques.

CP 3 : Développer une interface utilisateur web dynamique pilotée par la donnée

- **Logique de rendu Thymeleaf (SpEL)** : Utilisation avancée du langage d'expression de Thymeleaf pour manipuler dynamiquement le DOM côté serveur. La structure de la page s'adapte automatiquement aux données métier (scores, listes d'offres, notifications) avant l'envoi au navigateur.
- **Injection dynamique de données** : Implémentation d'un rendu conditionnel via les attributs **th:if** et **th:switch** pour gérer l'affichage en temps réel des alertes de succès, des erreurs et de l'état des candidatures.
- **Interactivité via Bootstrap Data-Attributes** : Utilisation des standards Bootstrap (data-bs-toggle) pour orchestrer l'ouverture des modales de détails et les interactions utilisateur, garantissant une interface fluide et réactive sans script personnalisé redondant.
- **Moteur de rendu sécurisé (SSR)** : Centralisation du calcul des scores de matching côté serveur. Cette approche de **Server-Side Rendering** garantit une sécurité maximale contre les manipulations côté client et une rapidité d'affichage optimale pour l'utilisateur final.

CP 4 : Réaliser une interface utilisateur avec un système de gestion de contenu ou une solution e-commerce

- Gestion de contenu sur-mesure (Back-Office personnalisé) : Bien que développée sans CMS type WordPress, l'application intègre un système complet de gestion de contenu autonome.
- Dashboard Entreprise : Conçu comme une tour de contrôle, les candidats y sont ordonnés nativement selon la valeur décroissante de leur score_matching. Un accès direct permet de télécharger le CV physique ou de consulter le portfolio en ligne du candidat.
- Dashboard Candidat : Met l'accent sur la transparence avec un feedback visuel immédiat présentant l'état d'avancement de la candidature, permettant au postulant de suivre l'évolution de son dossier.
- Cycle de vie des candidatures : Workflow strict avec les statuts EN_ATTENTE, ACCEPTE, REFUSE et ENTRETIEN, reflétant la réalité métier du recrutement.

ACTIVITÉ TYPE 2 : Développer la partie back-end d'une application web ou web mobile sécurisée

CP 5 : Créer une base de données

- Architecture de stockage polyglotte : Modélisation et déploiement de deux systèmes de bases de données distincts, adaptés à la nature des données :
 - MySQL 8 (Relationnel) : Déploiement d'une base relationnelle SQL structurée pour le noyau applicatif :
 - Gestion rigoureuse des comptes utilisateurs (users).
 - Profils candidats (candidats) et entreprises (entreprises).
 - Offres d'emploi (offres).
 - Candidatures (candidatures) et entretiens (entretiens).
 - Expériences professionnelles (experiences).
 - MongoDB 6.0 (NoSQL orienté Documents) : Choix technologique spécifique pour le stockage des :
 - Notifications (historique des alertes pour candidats, administrateurs et entreprises).
- Méthode Merise : Modélisation de la base de données à travers trois niveaux d'abstraction :
 - MCD (Modèle Conceptuel de Données) : Définition des règles métier (ex : un candidat ne peut avoir qu'un seul CV, mais peut postuler à plusieurs offres).
 - MLD (Modèle Logique de Données) : Adaptation du MCD à l'outil informatique MySQL (tables de jointure comme candidatures).
 - MPD (Modèle Physique de Données) : Code SQL final définissant les types précis (VARCHAR, TEXT) et les contraintes techniques (NOT NULL, AUTO_INCREMENT).
- Conformité RGPD par la conception (Privacy by Design) : Implémentation des clauses ON DELETE CASCADE garantissant que la suppression d'un compte utilisateur efface proprement toutes ses données liées (CV, candidatures), répondant ainsi au droit à l'oubli exigé par le RGPD.

CP 6 : Développer les composants d'accès aux données

- Composants de Persistance SQL (Spring Data JPA) :
 - Implémentation de l'ORM Hibernate via Spring Data JPA.
 - Utilisation des annotations standard (@Entity, @Table, @Id, @ManyToOne, @OneToMany) pour mapper le Modèle Conceptuel de Données.
 - Abstraction des requêtes SQL via des interfaces Repository propres.
 - Protection native contre les injections SQL grâce à l'utilisation systématique de requêtes préparées.
- Composants de Persistance NoSQL (Spring Data MongoDB) :
 - Utilisation de @Document(collection = "notifications") pour la persistance des notifications.
 - Manipulation fluide des documents via MongoRepository.
 - Requetes spécifiques (ex : récupération des notifications non lues triées par ordre antéchronologique).
- Gestion des rôles et permissions (Principe du Moindre Privilège) :
 - Compte root dédié à l'administration globale (Droits DDL).
 - Utilisateur applicatif skillmatchingApp disposant uniquement des droits DML (SELECT, INSERT, UPDATE, DELETE).
 - Séparation minimisant la surface d'attaque en production.

CP 7 : Développer la partie back-end d'une application web ou web mobile :

- Architecture Microservices découplée : Conception et découpage du projet en cinq services indépendants via Maven, garantissant une séparation stricte des responsabilités :
 - **skillmatch-auth-service** : Gestion de l'identité, chiffrement par hachage (BCrypt) et émission des jetons d'accès JWT.
 - **skillmatch-candidat-service** : Encapsulation de la logique d'extraction et de structuration des informations des candidats.
 - **skillmatch-entreprise-service** : Gestion du cycle de vie des profils d'entreprises.
 - **skillmatch-offres-service** : Publication, modification et clôture des fiches d'offres.
 - **skillmatch-candidature-service** : Logique d'alignement algorithmique et persistance des scores de matching.
 - **skillmatch-api-gateway** : Point d'entrée unique (Reverse Proxy) centralisant le routage dynamique et la sécurité transversale.
 - **frontend-skillmatch** : Couche d'interface utilisateur découplée communiquant exclusivement avec l'API Gateway.
- Architecture en Couches (N-Tiers) au sein de chaque microservice :
 - **Couche Présentation/Contrôleur** : Gestion des points d'accès REST, validation des DTO via @Valid, capture des exceptions.
 - **Couche Métier (Service)** : Sanctuaire des règles d'affaires et de l'algorithme de matching.

- **Couche Accès aux Données (Repository)** : Gestion exclusive de l'infrastructure physique et du mapping (SQL ou Documentaire).
- Découplage absolu et Jointures Applicatives :
 - Respect strict de l'isolation des services : aucune jointure physique SQL cross-service.
 - HashiCorp Consul agit comme annuaire de services (Service Discovery) pour localiser dynamiquement chaque module sur le réseau.
 - OpenFeign implémenté pour orchestrer les échanges HTTP inter-services avec propagation du contexte de sécurité JWT.
- API REST standardisée : séparation stricte entre Frontend et Backend via l'échange exclusif de données standardisées au format JSON, permettant l'évolution future vers une application mobile native.

CP 8 : Élaborer et mettre en œuvre des composants dans une application de gestion de contenu ou e-commerce :

- Composants de Sécurité Globale (Spring Security) :
 - Configuration d'un pare-feu applicatif robuste avec authentification Stateless par jeton JWT.
 - Chiffrement des mots de passe par hachage adaptatif BCrypt (facteur de force 12).
 - Contrôle d'accès granulaire basé sur les rôles (RBAC) : ROLE_CANDIDAT, ROLE_ENTREPRISE, ROLE_ADMIN.
 - Interception des tentatives d'accès sans privilèges (ex : candidat accédant au Dashboard Admin) renvoyant une erreur HTTP 403 Forbidden.
- Protection contre les failles OWASP majeures :
 - SQL Injection : Spring Data JPA (requêtes préparées).
 - XSS (Cross-Site Scripting) : En-têtes sécurisés, échappement automatique.
 - CSRF : JWT (stateless) + CSRF désactivé.
 - Path Traversal : Validation des chemins de fichiers.
 - Mass Assignment : DTOs (pas d'exposition directe des entités).
- Composants Transverses & Intercepteurs :
 - Implémentation d'un intercepteur personnalisé intégré au cycle de vie OpenFeign pour propager automatiquement le jeton JWT dans les requêtes inter-services.
 - GlobalExceptionHandler pour la capture et le formatage standardisé des erreurs API.
 - Filtre JwtAuthenticationFilter extrayant les autorités (GrantedAuthority) et initialisant le SecurityContext Spring de manière éphémère.
- Conformité RGPD :
 - Droit à l'information : Politique de confidentialité sur le site.
 - Droit d'accès : Endpoint GET /api/users/{id}.
 - Droit de rectification : Endpoint PUT /api/users/{id}.
 - Droit à l'effacement : Soft delete + anonymisation en cascade.

COMPÉTENCE TRANSVERSE : Conteneurisation & Orchestration

Docker & Docker-Compose :

- **Builds multi-étapes** : Les fichiers Dockerfile séparent hermétiquement l'environnement de compilation de l'environnement d'exécution final :
 - Étape de construction : Environnement complet (Maven, JDK 17) pour compiler le code et générer le fichier .jar.
 - Phase d'exécution : Environnement Java épuré (slim), excluant le code source et les outils de build.
 - Avantages : Réduction de la surface d'attaque en production, minimisation du poids des images Docker.
- **Orchestration réseau (Docker Compose)** : Un unique fichier docker-compose.yml pilote et monte simultanément :
 - L'instance MySQL 8 (port interne 3306 mappé sur 3307 hôte).
 - L'instance MongoDB 6.0 (port interne 27017 mappé sur 27018 hôte).
 - Le serveur de découverte HashiCorp Consul.
 - Les cinq microservices métiers.
 - L'API Gateway.
- **Réseau interne isolé** : Création d'un sous-réseau privé skillmatch-network en mode bridge. Les microservices communiquent directement via ce canal sans exposer leurs ports transactionnels vers l'extérieur.
- **Stratégie de persistance des données** : Utilisation de volumes nommés externes (mysql_data et mongo_data) garantissant la persistance des données utilisateur (comptes, offres, historiques de notifications) lors de l'arrêt, la mise à jour ou le redémarrage des conteneurs.
- **Découverte de services** : **Consul** émet des requêtes de vérification d'état (Health checks) régulières sur l'endpoint /actuator/health de chaque conteneur, assurant la haute disponibilité de l'application.

Conclusion Générale

Le développement complet de l'infrastructure décentralisée SkillMatch valide de manière formelle l'acquisition des compétences transversales requises par le titre professionnel Développeur Web et Web Mobile (DWWM).

Ce projet démontre la capacité à :

- Transformer une fonction complexe (le tri sémantique de candidatures) en un système distribué hautement disponible, sécurisé et évolutif.
- Maîtriser le stockage de données moderne avec une persistance polyglotte (MySQL 8 + MongoDB).
- Implémenter une sécurité applicative robuste (JWT, BCrypt, RBAC) conforme aux standards OWASP et RGPD.
- Déployer une architecture microservices conteneurisée avec Docker et orchestrée via Docker Compose.
- Créer une interface utilisateur responsive, fluide et accessible avec Thymeleaf, Tailwind CSS et Bootstrap.

Verrous Techniques Résolus

Le principal défi technique a concerné la propagation du contexte de sécurité (jetons JWT) à travers la passerelle API et les appels inter-services OpenFeign. La solution a consisté à développer un intercepteur personnalisé extrayant dynamiquement le jeton JWT du contexte de sécurité actif pour l'injecter automatiquement dans les en-têtes des requêtes sortantes, garantissant la continuité du contrôle d'accès RBAC de bout en bout.

Perspectives d'Évolution

1. **Transition vers une Architecture Orientée Événements (Apache Kafka) :** Remplacer les appels HTTP synchrones par un modèle asynchrone résilient, permettant l'absorption de pics de charge sans latence pour l'utilisateur.
2. **Optimisation algorithmique par Intelligence Artificielle (NLP) :** Intégrer un moteur d'analyse sémantique poussé basé sur le traitement naturel du langage et des modèles de vectorisation de embeddings, permettant à SkillMatch de comprendre les concepts et métiers à proximité.
3. **Industrialisation de la documentation (Swagger/OpenAPI 3) :** L'intégration de Swagger permettrait de générer automatiquement une documentation interactive des APIs.
4. **Mise en place d'un module de communication temps réel (Chat & Forum) :** Afin de favoriser l'interaction directe et la transparence entre les recruteurs et les talents.

Bibliographie & Références

- Spring Boot Documentation : <https://spring.io/projects/spring-boot>
- Spring Cloud Documentation : <https://spring.io/projects/spring-cloud>
- Resilience4j Documentation : <https://resilience4j.readme.io/>
- OWASP Top 10 : <https://owasp.org/Top10/>
- <https://www.baeldung.com/>
- <https://www.javacodegeeks.com/>
- <https://stackoverflow.com/>
- <https://www.youtube.com/@mohamedYoussfi>